Bidirectional Encoder Representations from Transformers

BERT, short for **B**idirectional **E**ncoder **R**epresentations from **T**ransformers, is a state-of-the-art natural language processing (NLP) model introduced by Devlin et al. in 2018. BERT's key innovation is its use of bidirectional context, allowing it to generate word representations that take into account the full context of a word from both its left and right.

Transformer Architecture

BERT is built on the Transformer architecture, specifically utilizing the encoder component. Introduced by Vaswani et al. in 2017, the Transformer relies on self-attention mechanisms, which replace the need for recurrent layers used in earlier NLP models.

Self-Attention Mechanism

The self-attention mechanism enables the model to weigh the importance of each word relative to others in a sequence. For an input sequence $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n]$, self-attention computes a new representation for each token by attending to all tokens:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

where:

- $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$: Queries.
- $\mathbf{K} = \mathbf{X}\mathbf{W}^K$: Keys.
- $\mathbf{V} = \mathbf{X}\mathbf{W}^V$: Values.
- d_k: Dimension of the key vectors.
- $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$: Learned weight matrices.

Multi-Head Attention

To capture different aspects of relationships between words, BERT employs multi-head attention, allowing multiple self-attention layers to run in parallel:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O$$

where each head i computes:

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$$

Position-wise Feed-Forward Networks

After the multi-head attention, BERT applies a feed-forward network (FFN) to each token:

$$\text{FFN}(\mathbf{x}) = \text{ReLU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

This enables complex non-linear transformations of the attended information.

BERT's Bidirectional Training

Unlike traditional models that predict words based on a unidirectional context, BERT uses bidirectional training, considering both preceding and succeeding words. This approach helps in capturing nuanced language patterns.

Training Objectives

BERT is pre-trained using two tasks:

Masked Language Modeling (MLM)

In MLM, random tokens in a sequence are masked, and the model is trained to predict these masked tokens:

$$\mathcal{L}_{MLM} = -\sum_{m \in \mathcal{M}} \log P(\mathbf{x}_m | \mathbf{X}')$$

where $\mathbf{X}'$ is the masked input sequence and $\mathcal{M}$ is the set of masked positions.

The Role of the [MASK] Token and Its Embedding

In BERT's masked language modeling (MLM) objective, certain tokens in the input sequence are replaced with the special token `[MASK]`. The goal of the model is to predict the original token at that position using the surrounding context.

Input Embedding of [MASK]. The `[MASK]` token is part of BERT's vocabulary and has a fixed, learned embedding vector $E_{[\text{MASK}]} \in \mathbb{R}^d$, where d is the hidden size (e.g., 768 for BERT-base). At the input stage, this embedding is the same across all sentences. That is,

$$\text{Embedding}(\texttt{[MASK]}) = E_{[\text{MASK}]}$$

regardless of sentence content.

Contextualization via Self-Attention. Though the initial embedding of `[MASK]` is fixed, BERT's transformer layers apply multiple rounds of multi-head self-attention. During this process, the hidden state corresponding to the `[MASK]` token is updated based on the surrounding words (i.e., its bidirectional context). Thus, the final representation of `[MASK]`, denoted $h_{\texttt{[MASK]}}$, varies from sentence to sentence:

$$h_{\texttt{[MASK]}} = \text{BERT}_{\text{final layer}}(\texttt{[MASK]})$$

Prediction Step. To predict the masked word, BERT passes the contextualized hidden state $h_{\texttt{[MASK]}}$ through a linear layer and a softmax over the vocabulary:

$$P(w \mid \text{context}) = \text{softmax}(W_o h_{\texttt{[MASK]}})$$

where $W_o \in \mathbb{R}^{|V| \times d}$ is the output projection matrix.

Key Insight. While `[MASK]` tokens begin with identical embeddings, their final hidden representations are *contextualized* based on neighboring tokens. This enables BERT to predict the original masked word using bidirectional context accurately.

- **Same input embedding:** `[MASK]` always maps to the same vector $E_{[\text{MASK}]}$.
- **Different contextual output:** $h_{\texttt{[MASK]}}$ changes depending on the sentence.
- **Full participation:** The `[MASK]` token is not ignored — it participates fully in attention layers.

Special Tokens in BERT: `[MASK]`, `[CLS]`, and Segment Embeddings

BERT introduces several special tokens and embedding mechanisms that play crucial roles in its architecture and pretraining objectives. Here, we describe the function and behavior of the `[MASK]` and `[CLS]` tokens, along with the use of segment embeddings.

The `[MASK]` Token. In masked language modeling (MLM), some input tokens are replaced with the special `[MASK]` token. This token is included in BERT's vocabulary and has a fixed, learnable embedding vector $E_{\texttt{[MASK]}} \in \mathbb{R}^d$. During pretraining, the model is trained to predict the original identity of these masked tokens based on context. While the embedding $E_{\texttt{[MASK]}}$ is always the same across all sentences, the final hidden state at that position, $h_{\texttt{[MASK]}}$, is contextualized by surrounding tokens via self-attention layers.

The `[CLS]` Token. The `[CLS]` token is prepended to every input sequence and is used for sentence-level tasks such as classification. Like `[MASK]`, it has a learned embedding $E_{\texttt{[CLS]}}$. After processing through all transformer layers, its final hidden state $h_{\texttt{[CLS]}}$ is interpreted as a summary representation of the entire sequence:

$$h_{\texttt{[CLS]}} = \text{BERT}_{\text{final layer}}(\texttt{[CLS]})$$

This vector is typically passed through additional layers (e.g., a feedforward network and softmax) for classification tasks such as sentiment analysis, natural language inference, and topic detection.

Segment Embeddings. To enable BERT to understand sentence relationships (e.g., in tasks like question answering or next sentence prediction), input sequences can consist of two segments, typically called Sentence A and Sentence B. Each token in the input is assigned a segment embedding:

- Tokens from Sentence A get segment embedding E_{seg_0}.
- Tokens from Sentence B get segment embedding E_{seg_1}.

The final embedding for each token is computed as:

$$\text{InputEmbedding}_i = E_{\text{token}_i} + E_{\text{position}_i} + E_{\text{segment}_i}$$

where:

- E_{token_i} is the word embedding,
- E_{position_i} is the positional encoding,
- E_{segment_i} is either E_{seg_0} or E_{seg_1}.

Segment embeddings help BERT distinguish between tokens from different sentences, especially during tasks where sentence relationships matter.

Summary of Special Embeddings

- `[MASK]`: Used for masked language modeling; same embedding for all sentences, but context-dependent hidden state.
- `[CLS]`: Used for sequence-level classification tasks; its hidden state represents the whole input.
- **Segment embeddings:** Allow BERT to distinguish between multiple sentences in one input.

Next Sentence Prediction (NSP)

NSP trains the model to predict whether a given sentence B follows sentence A:

$$\mathcal{L}_{NSP} = -\sum_{(A,B)} [y \log P(\text{IsNext}|A,B) + (1-y)\log(1 - P(\text{IsNext}|A,B))]$$

where y is 1 if B follows A and 0 otherwise.

Fine-Tuning BERT

After pre-training, BERT can be fine-tuned for various tasks by adding a task-specific layer and training on labeled data:

$$\mathcal{L} = -\sum_{i=1}^{N} y_i \log(\hat{y}_i)$$

where y_i is the true label and $\hat{y}_i$ is the predicted probability.

BERT's Impact and Applications

BERT has been instrumental in improving various NLP tasks, such as:

- **Question Answering**: Extracting answers from text, as demonstrated on datasets like SQuAD.
- **Text Classification**: Tasks such as sentiment analysis and spam detection.
- **Named Entity Recognition (NER)**: Identifying entities like names and dates in text.
- **Machine Translation**: Improving translations by understanding context better.

Mathematical Example: Self-Attention Calculation

To illustrate the self-attention mechanism, consider an input sequence of three tokens with embeddings of dimension $d = 2$:

$$\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

with learned weight matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, and $\mathbf{W}^V$. Compute the query, key, and value matrices, calculate attention scores, apply softmax, and obtain attended representations, demonstrating how each word attends to others in the sequence.

Advantages of BERT

- **Contextual Understanding**: Captures word meanings based on context.
- **Transfer Learning**: Can be fine-tuned for new tasks with relatively small datasets.
- **Versatility**: Adapts to a range of NLP applications.

Extensions and Variants of BERT

Numerous BERT variants aim to enhance its capabilities:

- **RoBERTa**: Optimizes BERT with improved training methods.
- **DistilBERT**: A more efficient, smaller version using knowledge distillation.
- **ALBERT**: Reduces parameter size by sharing layers.
- **SpanBERT**: Focuses on span-level predictions for tasks like QA.

BERT in Multi-Modal Learning

BERT can be combined with models handling other modalities (e.g., images) to tackle tasks like image captioning or visual question answering (VQA), where textual and visual inputs are jointly processed to enhance understanding and generate relevant outputs.